

GitHub Verified Commit

Requirements: -

Verified Commit: -

1. The commit must be cryptographically signed (GPG or SSH signing).
2. The public key used to sign the commit must be added to your GitHub account.
3. The email in the commit (`Author:`) must match an email that's verified on your GitHub account (or be covered by the uploaded key's UID).

Steps:

Ubuntu: -

1. `sudo apt update`
2. `sudo apt upgrade`
3. `gpg`
4. `gpg --full-generate-key`

- **Kind of key:** choose `(1) RSA and RSA`
- **Keysize:** `4096` (strong)
- **Expiry:** enter `0` for never expire or `1y / 2y`, etc. (choose your preference)
- **Real name:** Your full name (use the name you want shown on commits)
- **Email address:** Use the **exact email** that is verified on your GitHub account. (If you use a GitHub no-reply email, you can use that — see GitHub settings.)
- **Comment:** optional — usually leave blank.
- **Passphrase:** choose a strong passphrase (recommended) — this protects your private key.

5. `gpg --list-secret-keys --keyid-format LONG`

```
yaml                                                                    Copy code

sec  rsa4096/ABC1234DEF567890 2025-11-02 [SC]
     Key fingerprint = 1234 5678 9ABC DEF0 1234 5678 9ABC DEF0 ABC1 2345
uid   Your Name <your.email@example.com>

• The long key ID is the part after the slash: ABC1234DEF567890 .
• The fingerprint is the full 40-hex string (useful for verifying).

You'll use the KEYID in the next step.
```

6. `gpg --armor --export ABC1234DEF567890`

Copy the entire output (from -----BEGIN PGP PUBLIC KEY BLOCK----- to -----END PGP PUBLIC KEY BLOCK-----).

On GitHub:

1. Go to **Settings** → **SSH and GPG keys** → **New GPG key** (or directly: GitHub Settings → GPG keys).
2. Paste the exported public key and save.

GitHub will now be able to verify signatures made with that private key (when the commit email matches a verified email).

7. `git config --global user.signingkey ABC1234DEF567890`
8. `git config --global commit.gpgsign true`
9. `git config --global user.name "Your Name"`
10. `git config --global user.email your.email@example.com`

Then confirm your signing key and GPG settings:

```
bash
```

```
git config --global user.signingkey  
git config --global commit.gpgsign
```

11. `echo "Past commit test" > old.txt`
12. `git add old.txt`
13. `export GIT_AUTHOR_DATE="2020-01-15T10:30:00+05:30"`
14. `export GIT_COMMITTER_DATE="2020-01-15T10:30:00+05:30"`
15. `git commit -S -m "Past dated verified commit" --date="$GIT_AUTHOR_DATE"`
16. `git push origin main`

If not able to push

- `sudo apt install gh`
- `gh repo clone yash_singh`
- `gh auth login`
- `git push -u origin main`
- `git push origin main --force`

✓ Option B: Change (amend) the date of an existing commit

If you already have a commit you want to backdate and verify:

1. Set your desired date:

```
bash

export GIT_AUTHOR_DATE="2021-05-10T12:00:00+05:30"
export GIT_COMMITTER_DATE="2021-05-10T12:00:00+05:30"
```

2. Re-sign and amend it:

```
bash

git commit --amend --no-edit -S --date="$GIT_AUTHOR_DATE"
```

3. Force push (since you changed history):

```
bash

git push --force-with-lease origin main
```

Now the same commit will appear on GitHub as **Verified** but dated in the past.

💡 Optional Tip: Backdate multiple commits

If you need to rewrite several commits to have older dates (and all signed), use interactive rebase:

```
bash

git rebase -i HEAD~3 # for last 3 commits
```

Copy code

Mark the ones you want to edit with **e**, then for each one:

```
bash

export GIT_AUTHOR_DATE="2020-12-01T10:00:00+05:30"
export GIT_COMMITTER_DATE="$GIT_AUTHOR_DATE"
git commit --amend --no-edit -S --date="$GIT_AUTHOR_DATE"
git rebase --continue
```

Copy code

Finally:

```
bash

git push --force-with-lease origin main
```

Copy code